

Jorge Manuel Torre

Ashburn, VA • jorgetorre06@gmail.com • 571-888-4832 • linkedin.com/in/jmt1006 • github.com/jmtorr3 • jmtorr3.github.io

Education

Virginia Tech

Blacksburg, VA

B.S. Computer Science, Overall GPA: 3.32/4.0, Major GPA: 3.50/4.0

May 2026

Relevant Coursework: Linux Kernel Programming, Software Reverse Engineering, Security of AI, Computer Architecture, Computer Systems, Data Structures & Algorithms.

Projects & Experience

Pokémon Black (Nintendo DS) Reverse Engineering

December 2024 – January 2025

ARM Assembly, DeSmuME, No\$GBA, PKHeX, Action Replay

- Decoded the Action Replay opcode and prefix format, then used working cheats as known-good memory patches to trace addresses tied to specific in-game behavior.
- Mapped player movement speed to a live ARM instruction at 0x02197FC4 (MOV R1, #0x0C) and reverse engineered the wild encounter system to a preloaded slot table at 0x02250010, verified by injecting species IDs and observing battle outcomes.
- Documented the Gen 5 in-RAM Pokémon data layout by cross-checking RAM dumps against PKHeX: 220-byte encrypted blocks containing species, IVs/EVs, nickname, OT, and nature, preceded by the N.pokeparty.c header.

Linux Kernel Programming (CS 4224, Virginia Tech)

January 2026 – April 2026

C, x86-64 Assembly, GDB, Linux kernel, BPF, sched_ext, seccomp

- Implemented custom syscalls (`sys_kstats`, `sys_procinfo`) in the Linux kernel with safe user/kernel data transfer; benchmarked overhead with `rdtsc/rdtscp` and analyzed vDSO fast paths against trapping syscalls.
- Built a kernel module instrumenting the EEVDF scheduler and traced `vruntime` evolution via `/proc`; ran fairness experiments across nice values, sleeper credit, and slice scaling.
- Implemented `lkp_sched_wrr`, a weighted round-robin scheduler, in `sched_ext` / **BPF**; compared latency and throughput against CFS/EEVDF on multi-core workloads.
- Hardened processes with `seccomp-bpf` filters and experimentally verified KPTI page-table isolation by inspecting CR3 transitions in GDB.
- Built a kernel memory profiler that walks page tables, samples 8-bit access counters from a scanner thread, applies exponential decay, classifies hot and cold pages, and exports reports through `/proc`.
- Implemented `lkpfs`, an in-memory Linux pseudo-filesystem module that exposes a live process tree through VFS registration, superblocks, inodes, dentries, `seq_file`, and inode eviction.

Computer Systems (CS 3214, Virginia Tech)

C, POSIX APIs, pthreads, sockets, HTTP/1.1, JWT

- Built a Unix shell with pipelines, I/O redirection, background jobs, process groups, signals, variable expansion, command substitution, and control-flow constructs.
- Implemented a concurrent HTTP/1.1 server with persistent connections, static file serving, MP4 streaming, JWT-based authentication, and path traversal checks.
- Wrote a fork/join thread pool with futures, per-worker dequeues, work stealing, and inline future execution to avoid starvation and deadlock in recursive workloads.
- Implemented `malloc`, `free`, and `realloc` for a dynamic memory allocator and added a heap consistency checker for debugging allocator invariants.

Experimental Home Lab

January 2021 – Present

Proxmox, OPNsense, Debian, Ubuntu, NixOS, Docker

- Run a multi-VM Proxmox lab for self-hosted services and isolated systems experiments, with OPNsense firewall rules and an SSH jumpbox for remote administration.
- Maintain a dedicated NixOS workspace for kernel development, testing, and a long-running Linux internals journal.

Skills

Languages: C, C++, Assembly (ARM, x86-64, RISC-V), Python, Java, SQL, JavaScript, Bash

Systems & Security: Reverse Engineering, Ghidra, Linux Kernel, Concurrency, ACID Transactions

Infrastructure: Proxmox, OPNsense, NixOS, Docker, PostgreSQL, Linux, Git, CI/CD